

Universidad Nacional de Colombia
Estructuras de Datos — 2026-2

Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Brallan Esteban Ardila Osorio
1140915162 · bardilao@unal.edu.co

Profesor: David Herrera
Monitora: Ángela Camila Siabato Londoño

Repositorio: <https://github.com/BrayanARDL/Laboratorio-stack-queue-java-ed->

Septiembre de 2026

Índice general

1	Objetivo	2
2	Explicación de la implementación	2
2.1	Organización del código	2
2.2	Listas enlazadas	2
2.3	Pila y cola con arreglo circular dinámico	3
2.4	Pruebas de correctitud	4
3	Metodología experimental	4
4	Análisis de complejidad	6
4.1	Hipótesis	6
4.2	Parte 1: comparativa de las implementaciones de List	6
4.3	Parte 2: MyStack y MyQueue con arreglo circular	12
4.3.1	Análisis del crecimiento del arreglo dinámico	14
4.4	Parte 3: comparativa de métodos equivalentes List / (Stack, Queue)	15
5	Conclusiones	17
5.1	Cuándo usar cada estructura	17
5.2	Pilas y colas en aplicaciones reales	17
5.3	Arreglos dinámicos vs. listas enlazadas en Java	17

1 Objetivo

Implementar en Java, sin usar las colecciones de la librería estándar, la estructura **List** con listas enlazadas en cuatro variantes (simplemente enlazada sin cola y con cola, doblemente enlazada sin cola y con cola) y las estructuras **MyStack** (pila) y **MyQueue** (cola) sobre un **arreglo circular dinámico**. Luego, analizar la complejidad de cada método de dos formas: *teóricamente*, con notación Big-O, y *empíricamente*, midiendo tiempos de ejecución con tamaños de 10^1 a 10^8 elementos. Con ambos análisis se comparan las implementaciones entre sí y se sacan conclusiones sobre cuándo conviene cada una.

El código fuente, los datos crudos de las mediciones y los scripts para reproducir todo están en el repositorio: <https://github.com/BrayanARDL/Laboratorio-stack-queue-java-ed->.

2 Explicación de la implementación

2.1 Organización del código

Paquete / archivo	Contenido
listas/LinkedListADT	Interfaz común de las cuatro listas (pushFront, pushBack, popFront, popBack, topFront, topBack, find, erase, addBefore, addAfter, isEmpty, size).
listas/SNode, DNode	Nodo simple (key, next) y nodo doble (key, next, prev).
listas/*	SinglyLinkedList, SinglyLinkedListTail, DoublyLinkedList, DoublyLinkedListTail.
pilacola/MyStack, MyQueue	Interfaces pedidas en el enunciado.
pilacola/CircularDynamicArray	Arreglo circular que crece duplicando su capacidad; es la base común.
pilacola/ArrayStack, ArrayQueue	Implementaciones de MyStack y MyQueue sobre el arreglo circular.
pruebas/Pruebas	Pruebas de correctitud (no miden tiempo).
benchmark/Benchmark	Medición de tiempos; escribe resultados/tiempos.csv.
analisis/graficar.py	Lee el CSV y genera las gráficas y tablas de este informe.

Las cuatro listas implementan la misma interfaz genérica `LinkedListADT<T, N>`, donde `N` es el tipo de nodo. Así el benchmark usa exactamente el mismo código de medición para las cuatro, y `find` puede devolver la *referencia* al nodo, como pide el enunciado, para usarla luego en `addBefore/addAfter`. Todas las estructuras guardan un contador `size`, que se actualiza en cada inserción y eliminación; por eso `size()` es $O(1)$ y no hay que recorrer la estructura.

2.2 Listas enlazadas

Operaciones al inicio. En las cuatro variantes `pushFront`, `popFront` y `topFront` solo tocan `head` y un par de referencias, así que son $O(1)$:

```
public void pushFront(T key) {           // SinglyLinkedList
    SNode<T> node = new SNode<>(key);
    node.next = head;                   // el nuevo apunta al antiguo primero
    head = node;                        // y pasa a ser el primero
    size++;
}
```

Operaciones al final. Sin referencia al último nodo (*sin cola*), `pushBack`, `topBack` y `popBack` tienen que recorrer toda la lista: $O(n)$. Guardar `tail` vuelve $O(1)$ a `pushBack` y `topBack`. Sin embargo, en la lista *simple con cola* `popBack` sigue siendo $O(n)$: al quitar el último, `tail` debe

pasar a apuntar al penúltimo, y un nodo simple no sabe quién lo antecede, así que hay que buscarlo desde `head`:

```
SNode<T> cur = head; // SinglyLinkedListTail.popBack()
while (cur.next != tail) cur = cur.next; // O(n): buscar el penúltimo
cur.next = null;
tail = cur;
```

Solo la lista *doble con cola* hace `popBack` en $O(1)$, porque `tail.prev` ya es el penúltimo.

Búsqueda y borrado. `find` recorre desde `head` hasta encontrar la clave y devuelve el nodo ($O(n)$ en el peor caso). `erase` también es $O(n)$: en la lista simple se busca el nodo *anterior* al que se borra para hacer `prev.next = prev.next.next`; en la doble se busca el propio nodo y se desenlaza en $O(1)$ usando `prev` y `next`.

Inserción relativa a un nodo. `addAfter(nodo, x)` es $O(1)$ en las cuatro, porque solo reacomoda referencias a partir del nodo recibido. `addBefore(nodo, x)` marca la diferencia: en listas simples hay que recorrer desde `head` buscando el anterior ($O(n)$); en listas dobles `nodo.prev` lo da directamente ($O(1)$):

```
DNode<T> nuevo = new DNode<>(key); // DoublyLinkedList.addBefore()
nuevo.next = node; nuevo.prev = node.prev;
if (node.prev != null) node.prev.next = nuevo; else head = nuevo;
node.prev = nuevo;
```

En las variantes con cola, cada método que puede cambiar el último elemento (`erase`, `addAfter`, `popBack`, vaciar la lista) actualiza también `tail`. Es la fuente de errores más común en estas estructuras, y por eso las pruebas de la sección 2.4 vacían y reutilizan cada lista.

2.3 Pila y cola con arreglo circular dinámico

`CircularDynamicArray` guarda los elementos en un arreglo `data`, con un índice `head` (posición física del primer elemento) y un contador `size`. El elemento lógico i vive en la posición física $(head + i) \bmod \text{capacidad}$, de modo que los elementos “dan la vuelta” al final del arreglo:

índice físico:	0	1	2	3	4	5	6	7
data:	e	f	.	.	a	b	c	d
			↑fin		↑head			

Sacar el primero solo avanza `head` ($O(1)$), sin desplazar nada. Esto es lo que hace eficiente a la cola: en un arreglo no circular, `dequeue` obligaría a correr todos los elementos una posición ($O(n)$).

- **ArrayStack:** la base es el inicio lógico y la cima el final. `push` = añadir al final, `pop` = quitar del final, `peek` = leer el final.
- **ArrayQueue:** `enqueue` = añadir al final, `dequeue` = quitar del inicio (avanza `head`), `front` = leer el inicio.
- **delete(n):** busca la primera aparición de n (desde la cima en la pila, desde el frente en la cola) y desplaza una posición los elementos que siguen, para no dejar huecos. Busca en $O(n)$ y desplaza en $O(n)$: en total $O(n)$.

Estrategia de crecimiento: duplicar la capacidad. Cuando el arreglo está lleno se crea uno del doble de tamaño y se copian los elementos en orden lógico a partir de la posición 0 (`System.arraycopy` en dos tramos, porque el contenido puede estar “partido” por la vuelta). Esa

inserción puntual cuesta $O(n)$, pero es $O(1)$ **amortizada**. Partiendo de capacidad c_0 , al insertar n elementos las copias ocurren con tamaños $c_0, 2c_0, 4c_0, \dots$ hasta n :

$$\text{copias totales} \leq c_0 + 2c_0 + 4c_0 + \dots + n < 2n \implies \frac{\text{costo total de } n \text{ inserciones}}{n} = \frac{O(n)}{n} = O(1).$$

Si en cambio el arreglo creciera de a c posiciones fijas, habría n/c redimensionamientos con copias de $c, 2c, 3c, \dots, n$ elementos:

$$\text{copias totales} = c(1+2+\dots+n/c) \approx \frac{n^2}{2c} \implies O(n^2) \text{ en total, } O(n) \text{ amortizado por inserción.}$$

Duplicar tiene un costo en memoria: justo después de crecer, la mitad del arreglo está vacía (se desperdicia hasta un 50 %). La implementación no reduce el arreglo cuando se vacía. Una mejora posible es reducirlo a la mitad cuando `size` baje a un cuarto de la capacidad, lo que conserva el costo amortizado $O(1)$. En la sección 4.3.1 se miden ambas estrategias.

2.4 Pruebas de correctitud

Antes de medir hay que asegurar que las estructuras funcionan. **pruebas.Pruebas** somete cada estructura a 200 000 operaciones aleatorias (con valores repetidos a propósito) y, después de cada una, la compara contra una lista de referencia de `java.util`. Revisa el valor devuelto, el tamaño, `isEmpty` y, periódicamente, el contenido completo. También comprueba que las operaciones sobre una estructura vacía lancen excepción y que cada lista siga funcionando después de vaciarse. Todas las estructuras pasan las pruebas, incluida la variante de crecimiento fijo. La librería estándar se usa *solo* como oráculo en las pruebas, nunca dentro de las estructuras.

3 Metodología experimental

Unidad de tiempo: nanosegundos. Se mide con `System.nanoTime()`. Una operación $O(1)$ tarda unas decenas de nanosegundos: medida en milisegundos (`Instant/toMillis`, como en el `Main.java` de guía) daría siempre 0 ms, y sería imposible distinguir, por ejemplo, un `popBack` $O(1)$ de uno $O(n)$ para listas de hasta 10^4 elementos. Con nanosegundos esa diferencia se ve desde $n = 100$. Para graficar, los tiempos se convierten a **microsegundos**, escala en la que tanto las operaciones rápidas (0,04 μ s) como las lentas (casi 10^6 μ s) se leen con pocos dígitos.

Resolución de la medición. Medir “nada” (dos llamadas seguidas a `nanoTime`) tarda entre 59 ns (mínimo) y 88 ns (percentil 90), con mediana de 73 ns, y la operación $O(1)$ más rápida medida tomó 34 ns en total. Es decir, el costo real de una operación $O(1)$ es de pocos nanosegundos, *por debajo* de lo que se puede medir de forma individual. Por eso todas las operaciones $O(1)$ aparecen en torno a 0,04–0,06 μ s: ese es el piso de la medición, no el costo de la operación. Lo que sí se puede afirmar con seguridad es que *no crecen* con n . El costo por operación sin el piso se estima en la sección 4.3.1, dividiendo el tiempo de n inserciones seguidas entre n .

Qué se mide exactamente. Para cada tamaño n se construye la estructura con n valores aleatorios en $[0, 10^9)$ y se mide **una sola operación**:

```
preparar.run();           // fuera de la medición
long t0 = System.nanoTime();
operacion.run();           // <- SOLO esto se mide
long t1 = System.nanoTime();
deshacer.run();           // fuera de la medición: la estructura vuelve a tener n
                           elementos
```

La fase *deshacer* garantiza que todas las repeticiones se hagan con exactamente n elementos. Por ejemplo, tras medir `pushBack` se ejecuta `popBack`, y tras `popFront` se vuelve a insertar el valor extraído. El programa verifica al final que el tamaño no haya cambiado.

Peor caso. Big-O describe el peor caso, así que ese es el que se mide:

Método	Caso medido
Find	Se busca un valor que no existe (centinela -1): recorre los n nodos.
Erase	El valor a borrar se inserta al final (fuera de la medición) y luego se borra: recorre todo.
AddBefore / AddAfter	Referencia al <i>último</i> nodo, obtenida con <code>find</code> fuera de la medición. Es el peor caso de <code>addBefore</code> en listas simples.
Push / Enqueue	Con espacio libre (caso típico). El caso con arreglo lleno se mide aparte (sec. 4.3.1).
Delete (pila)	El valor está en la <i>base</i> : se busca desde la cima recorriendo todo y luego se desplazan todos los de encima. Cada repetición trabaja sobre una copia recién hecha de la pila.
Delete (cola)	El valor está al <i>final</i> : la búsqueda desde el frente recorre los n elementos.

Repeticiones y calentamiento. La JVM compila el código “en caliente” (JIT), así que las primeras ejecuciones son más lentas. Por eso: (1) antes de medir se ejecuta todo el benchmark con tamaños pequeños sin guardar resultados; (2) en cada punto se descartan las primeras repeticiones. Se hacen 50 repeticiones para $n \leq 10^5$, 20 para 10^6 , 10 para 10^7 y 5 para 10^8 . Se reporta el **promedio recortado**: se descarta el 10 % más alto y el 10 % más bajo, lo que atenúa las pausas esporádicas del recolector de basura o del sistema operativo.

Separación entre medición y graficación. Los tiempos se guardan en memoria durante la medición y se escriben a un CSV solo al terminar cada serie. La graficación la hace un programa aparte en Python (`analisis/graficar.py`) que lee ese CSV, así que ni el dibujo ni la escritura a disco ocurren dentro del intervalo medido. Los datos crudos (18 334 mediciones) están en el repositorio.

Entorno. OpenJDK 21, Linux, procesador Intel Xeon a 2,1 GHz (2 núcleos), 7 GB de RAM. JVM ejecutada con `-Xms5g -Xmx5g -XX:+UseParallelGC -XX:+AlwaysPreTouch`: memoria fija y reservada desde el inicio (evita fallos de página durante la medición) y un recolector sin hilos concurrentes que compitan con la medición. Para que la lista de 10^8 nodos quepa en memoria, los valores aleatorios se toman de un banco precalculado de 2^{20} objetos `Integer`, en lugar de crear 10^8 objetos nuevos. Los valores siguen siendo aleatorios y se ahorra alrededor de 1,6 GB.

Cómo leer las gráficas y tablas. Ambos ejes están en escala logarítmica. En ella, una operación $O(1)$ es una línea horizontal y una $O(n)$ es una recta de pendiente 1: sube una década por cada década de n . Para comparar la teoría con lo medido, cada tabla incluye la **razón** $\times 10$, es decir, cuánto se multiplica el tiempo al pasar de $n = 10^7$ a $n = 10^8$: $\approx \times 1$ para $O(1)$, $\approx \times 10$ para $O(n)$ y $\approx \times 100$ para $O(n^2)$. Se usa la última década porque ahí ya se saturaron los efectos constantes de caché y del piso de medición, y domina el comportamiento asintótico. La fila *Empírico* clasifica esa razón (menor que $\times 3$: $O(1)$; entre $\times 3$ y $\times 30$: $O(n)$).

4 Análisis de complejidad

4.1 Hipótesis

Cuadro 1: Hipótesis: complejidad teórica en el peor caso.

Método	Simple sin cola	Simple con cola	Doble sin cola	Doble con cola	Arreglo circular
PushFront / Push	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)^*$
PushBack / Enqueue	$O(n)$	$O(1)$	$O(n)$	$O(1)$	$O(1)^*$
PopFront / Pop, Dequeue	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
PopBack	$O(n)$	$O(n)$	$O(n)$	$O(1)$	–
TopFront / Peek, Front	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
TopBack	$O(n)$	$O(1)$	$O(n)$	$O(1)$	–
Find	$O(n)$	$O(n)$	$O(n)$	$O(n)$	–
Erase / Delete	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
AddBefore	$O(n)$	$O(n)$	$O(1)$	$O(1)$	–
AddAfter	$O(1)$	$O(1)$	$O(1)$	$O(1)$	–
Empty / Size	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$

* Amortizado: la inserción que dispara el crecimiento cuesta $O(n)$.

Además de las clases de complejidad, se formulan tres hipótesis sobre las constantes:

- H1.** Entre variantes con la misma Big-O, las diferencias serán pequeñas: los métodos $O(1)$ deberían ser indistinguibles con la resolución disponible.
- H2.** Recorrer un arreglo será más rápido que recorrer una lista enlazada del mismo tamaño, porque el arreglo es contiguo en memoria y aprovecha la caché del procesador, mientras que en la lista cada nodo es un objeto aparte al que se llega siguiendo una referencia.
- H3.** Con incremento fijo, el costo total de n inserciones crecerá cuadráticamente; duplicando, crecerá linealmente.

4.2 Parte 1: comparativa de las implementaciones de List

Las tablas 2–5 muestran el tiempo promedio en microsegundos de cada método, y las figuras 1–4 lo grafican al estilo del ejemplo del enunciado. La figura 5 pone las cuatro implementaciones juntas, un panel por método, que es la vista más directa para comparar sin cola / con cola y simple / doble.

Cuadro 2: Lista simplemente enlazada **sin cola**: tiempo promedio (μ s).

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter
10^1	0,044	0,052	0,046	0,058	0,109	0,094	0,079	0,048
10^2	0,044	0,207	0,046	0,244	0,327	0,306	0,279	0,048
10^3	0,092	1,783	0,046	2,117	2,950	3,176	2,521	0,091
10^4	0,094	18,1	0,046	21,0	40,5	30,8	39,1	0,162
10^5	0,044	206,6	0,046	231,0	488,8	469,9	459,0	0,922
10^6	0,042	2 249	0,047	2 400	4 788	2 929	4 186	2,045
10^7	0,043	53 751	0,044	55 819	73 915	62 848	71 635	3,483
10^8	0,041	579 462	0,043	585 528	724 869	595 694	726 060	3,528
Teórico	$O(1)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Razón $\times 10$	$\times 1,0$	$\times 10,8$	$\times 1,0$	$\times 10,5$	$\times 9,8$	$\times 9,5$	$\times 10,1$	$\times 1,0$
Empírico	$O(1)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$

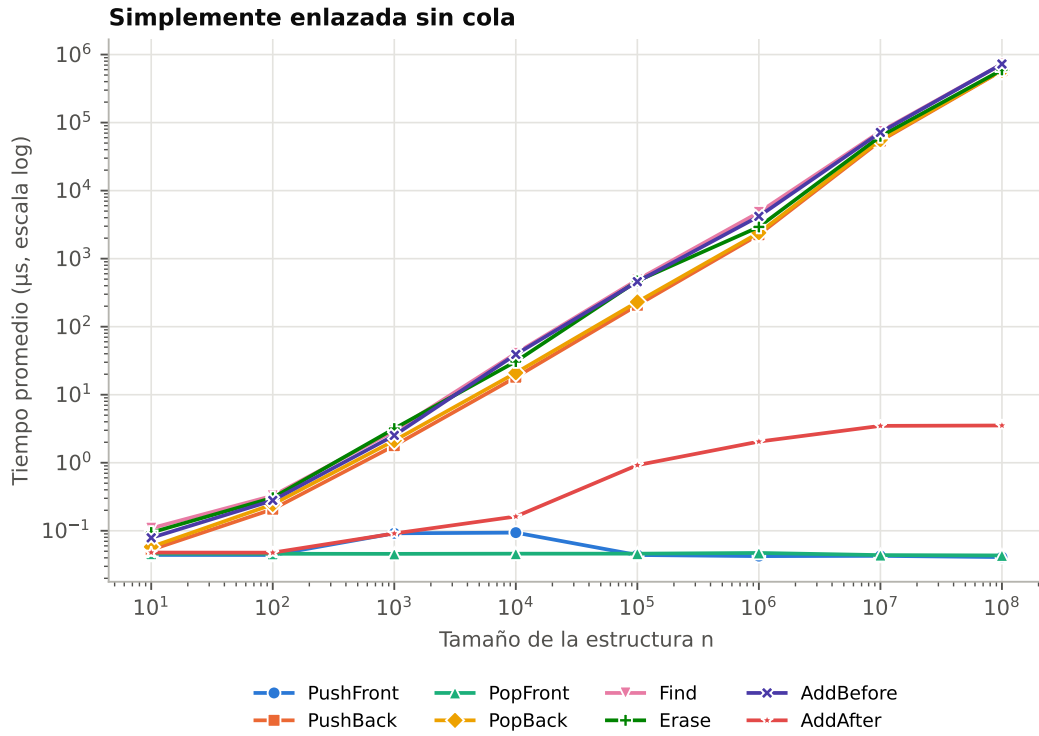


Figura 1: Lista simplemente enlazada sin cola.

Cuadro 3: Lista simplemente enlazada **con cola**: tiempo promedio (μ s).

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter
10^1	0,044	0,046	0,047	0,052	0,073	0,076	0,063	0,049
10^2	0,044	0,046	0,046	0,207	0,329	0,336	0,280	0,050
10^3	0,044	0,046	0,046	1,770	2,962	2,559	2,776	0,124
10^4	0,092	0,078	0,046	17,5	41,2	27,0	38,7	0,155
10^5	0,044	0,312	0,046	194,9	461,1	292,3	443,5	0,691
10^6	0,035	0,383	0,038	2 049	4 450	2 988	4 232	0,715
10^7	0,042	2,931	0,045	54 403	71 898	59 234	74 282	3,559
10^8	0,041	3,087	0,048	559 131	747 846	586 795	740 878	3,809
Teórico	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Razón $\times 10$	$\times 1,0$	$\times 1,1$	$\times 1,1$	$\times 10,3$	$\times 10,4$	$\times 9,9$	$\times 10,0$	$\times 1,1$
Empírico	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$

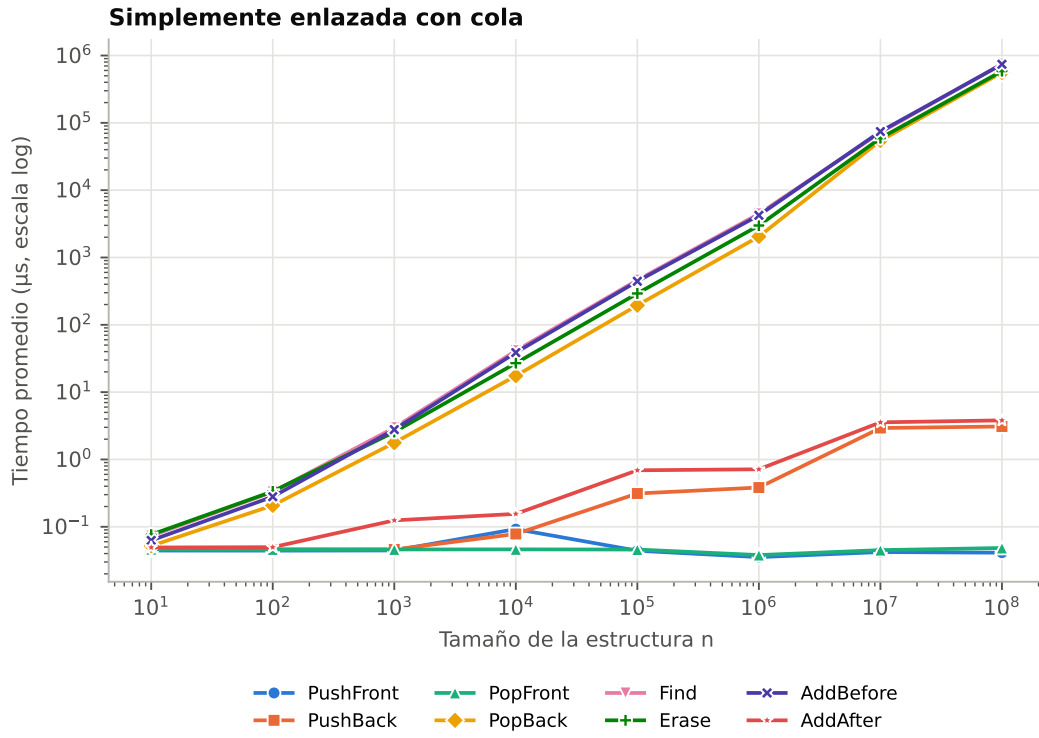


Figura 2: Lista simplemente enlazada con cola.

Cuadro 4: Lista doblemente enlazada **sin** cola: tiempo promedio (μs).

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter
10^1	0,060	0,076	0,058	0,075	0,101	0,196	0,173	0,193
10^2	0,045	0,212	0,046	0,216	0,329	0,343	0,048	0,047
10^3	0,054	1,843	0,060	1,780	3,561	2,860	0,087	0,083
10^4	0,045	17,9	0,046	18,1	43,4	43,6	0,152	0,161
10^5	0,057	206,5	0,046	195,4	486,4	485,8	0,613	0,763
10^6	0,043	2 194	0,044	2 232	4 986	4 659	1,413	1,329
10^7	0,042	56 374	0,045	58 664	76 445	76 156	2,810	2,872
10^8	0,044	596 021	0,048	580 340	757 307	788 777	3,157	3,047
Teórico	$O(1)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Razón $\times 10$	$\times 1,0$	$\times 10,6$	$\times 1,1$	$\times 9,9$	$\times 9,9$	$\times 10,4$	$\times 1,1$	$\times 1,1$
Empírico	$O(1)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$

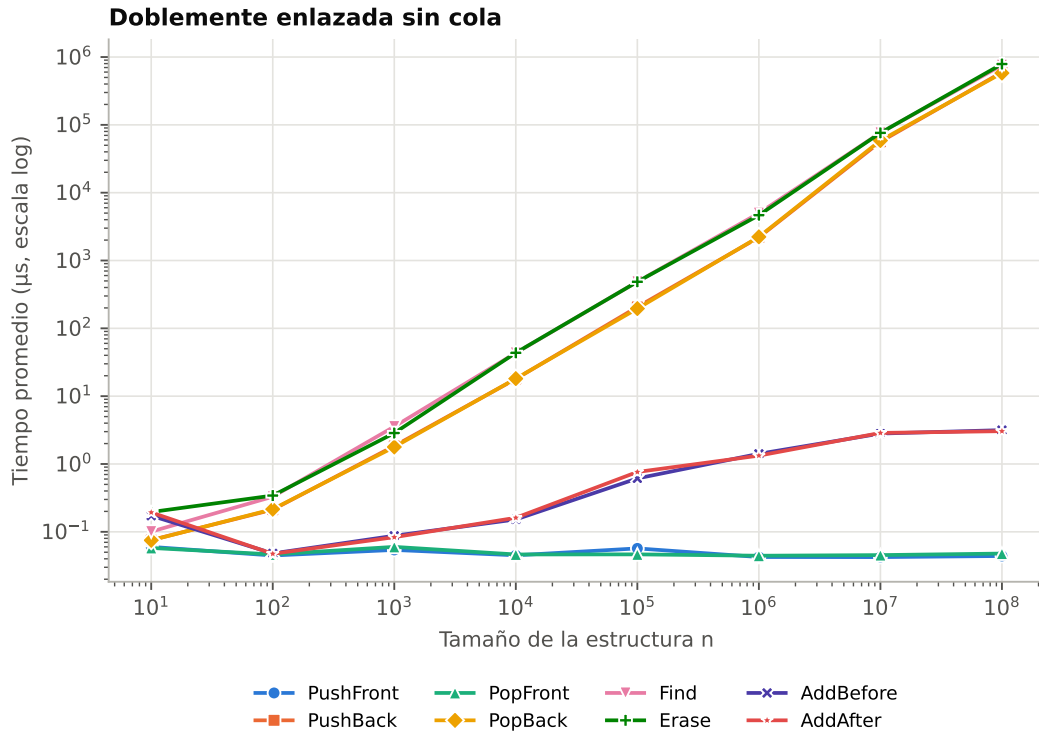


Figura 3: Lista doblemente enlazada sin cola.

Cuadro 5: Lista doblemente enlazada **con cola**: tiempo promedio (μs).

n	PushFront	PushBack	PopFront	PopBack	Find	Erase	AddBefore	AddAfter
10^1	0,044	0,047	0,049	0,050	0,078	0,139	0,124	0,120
10^2	0,045	0,047	0,049	0,049	0,329	0,399	0,098	0,118
10^3	0,045	0,047	0,049	0,050	3,009	3,249	0,148	0,188
10^4	0,044	0,047	0,049	0,049	40,8	41,4	0,151	0,343
10^5	0,039	0,047	0,061	0,049	451,3	464,9	0,582	2,543
10^6	0,043	0,046	0,045	0,045	4 580	4 447	1,031	3,862
10^7	0,043	0,045	0,046	0,045	69 954	72 049	3,279	17,2
10^8	0,042	0,046	0,046	0,044	935 985	949 247	3,703	4,313
Teórico	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Razón $\times 10$	$\times 1,0$	$\times 1,0$	$\times 1,0$	$\times 1,0$	$\times 13,4$	$\times 13,2$	$\times 1,1$	$\times 0,3$
Empírico	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$

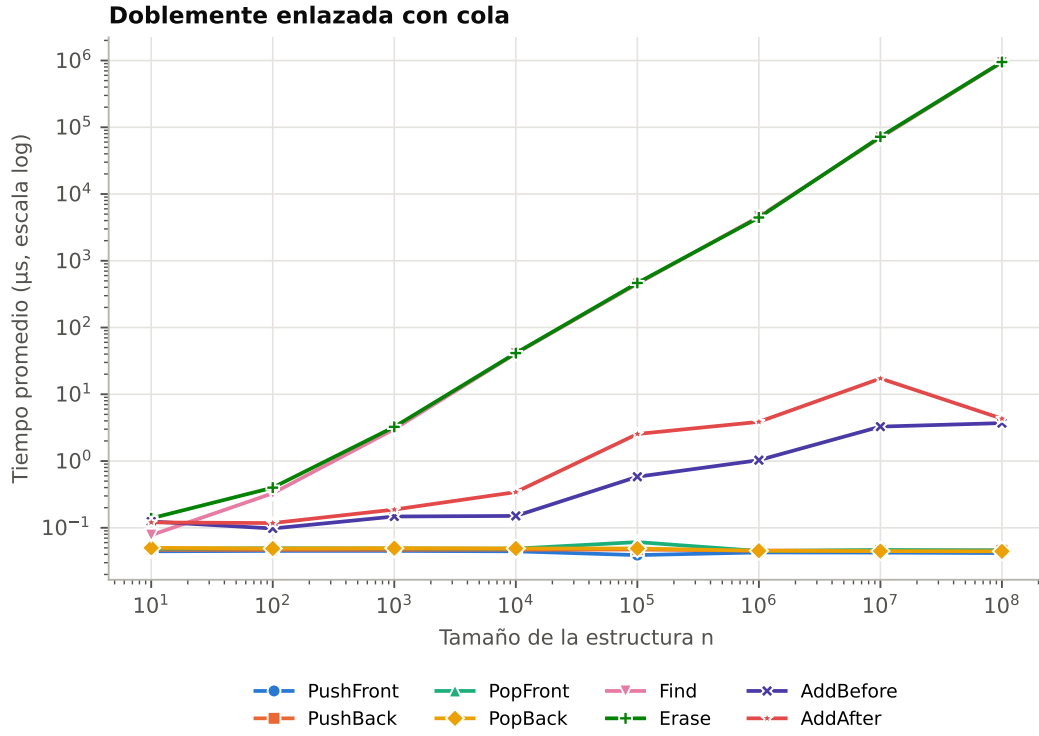


Figura 4: Lista doblemente enlazada con cola.

Cuadro 6: Métodos auxiliares (TopFront, TopBack, Empty, Size) de las cuatro listas, μs .

n	TopFront	TopBack	Empty	Size	n	TopFront	TopBack	Empty	Size
10^1	0,051	0,059	0,051	0,052	10^1	0,052	0,052	0,052	0,053
10^2	0,052	0,215	0,052	0,053	10^2	0,052	0,052	0,052	0,053
10^3	0,052	1,801	0,052	0,057	10^3	0,052	0,052	0,052	0,057
10^4	0,052	17,7	0,052	0,056	10^4	0,052	0,052	0,052	0,057
10^5	0,068	210,8	0,052	0,054	10^5	0,052	0,052	0,051	0,053
10^6	0,052	2 102	0,052	0,055	10^6	0,043	0,043	0,043	0,044
10^7	0,049	55 605	0,049	0,050	10^7	0,050	0,050	0,050	0,050
10^8	0,050	565 734	0,064	0,068	10^8	0,051	0,050	0,054	0,055
Teórico	$O(1)$	$O(n)$	$O(1)$	$O(1)$	Teórico	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Razón $\times 10$	$\times 1,0$	$\times 10,2$	$\times 1,3$	$\times 1,4$	Razón $\times 10$	$\times 1,0$	$\times 1,0$	$\times 1,1$	$\times 1,1$
Empírico	$O(1)$	$O(n)$	$O(1)$	$O(1)$	Empírico	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Simple sin cola					Simple con cola				
n	TopFront	TopBack	Empty	Size	n	TopFront	TopBack	Empty	Size
10^1	0,058	0,074	0,088	0,060	10^1	0,052	0,052	0,077	0,053
10^2	0,052	0,217	0,052	0,053	10^2	0,052	0,052	0,076	0,053
10^3	0,051	1,776	0,051	0,057	10^3	0,052	0,052	0,076	0,057
10^4	0,052	17,7	0,052	0,057	10^4	0,052	0,052	0,058	0,057
10^5	0,057	206,8	0,052	0,054	10^5	0,052	0,052	0,052	0,053
10^6	0,052	2 161	0,049	0,050	10^6	0,050	0,050	0,049	0,050
10^7	0,050	58 277	0,049	0,050	10^7	0,050	0,050	0,049	0,049
10^8	0,050	583 623	0,052	0,051	10^8	0,050	0,051	0,054	0,051
Teórico	$O(1)$	$O(n)$	$O(1)$	$O(1)$	Teórico	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Razón $\times 10$	$\times 1,0$	$\times 10,0$	$\times 1,1$	$\times 1,0$	Razón $\times 10$	$\times 1,0$	$\times 1,0$	$\times 1,1$	$\times 1,0$
Empírico	$O(1)$	$O(n)$	$O(1)$	$O(1)$	Empírico	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Doble sin cola					Doble con cola				

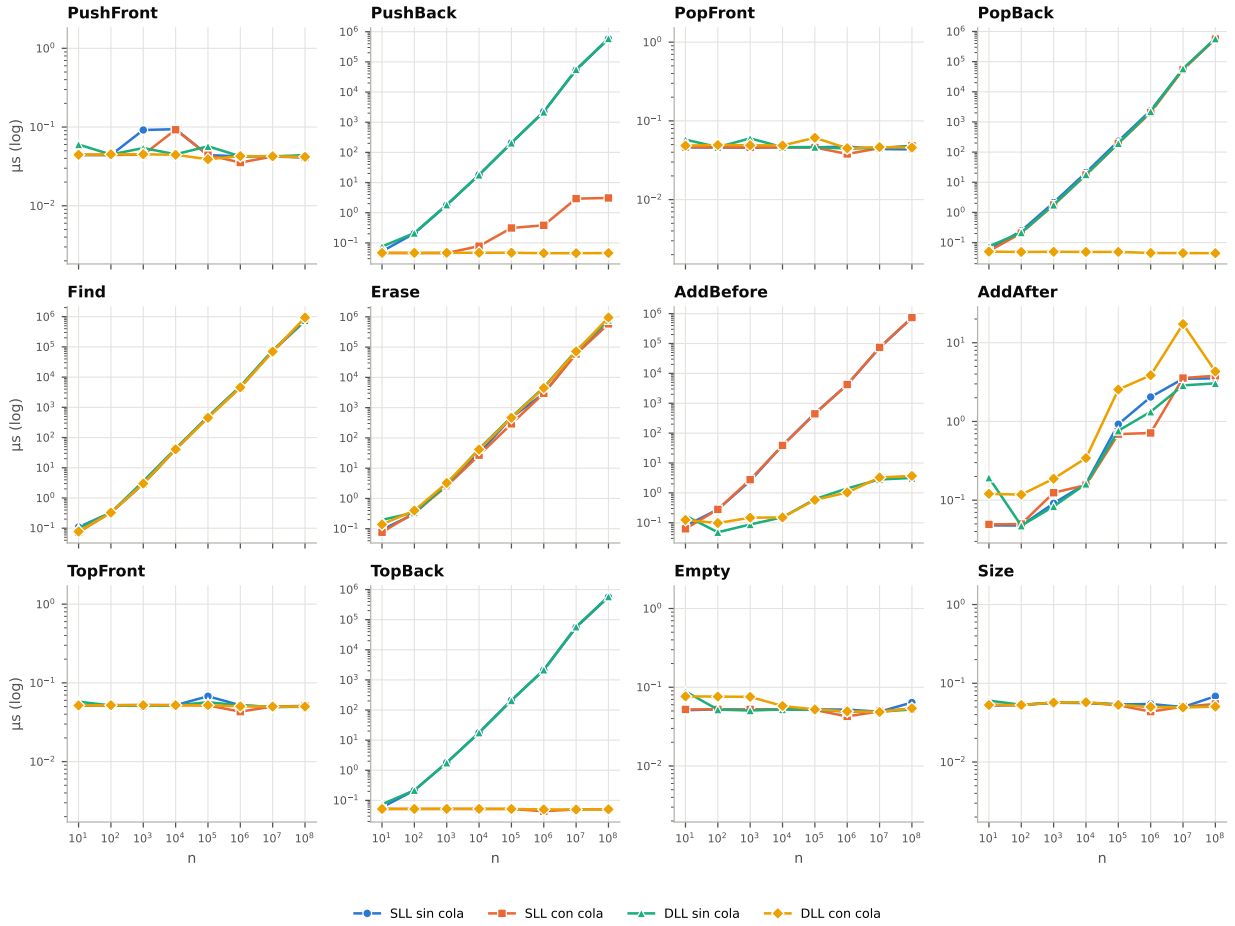


Figura 5: Cada método comparado entre las cuatro implementaciones.

Teórico vs. empírico. En las cuatro implementaciones y los doce métodos, la clasificación empírica coincide con la teórica. Todos los métodos $O(n)$ multiplican su tiempo por ≈ 10 entre 10^7 y 10^8 (razones entre $\times 9,5$ y $\times 13,4$), y todos los $O(1)$ se mantienen prácticamente constantes. Los casos más ilustrativos:

- **PopBack** es el método que mejor separa las variantes. Sin cola, o con cola pero simple, vale 0,56–0,59s para 10^8 elementos; en la doble con cola vale 0,044 μs : unos **13 millones de veces menos**, y constante desde $n = 10$. Esto confirma la explicación de la sección 2: a la lista simple con cola no le basta con **tail**, le hace falta **prev**.
- **PushBack** y **TopBack** pasan de $O(n)$ a $O(1)$ en cuanto la lista guarda **tail**, sea simple o doble.
- **AddBefore** separa simples de dobles, sin importar la cola: 0,73–0,74s en las simples contra 3–4 μs en las dobles para $n = 10^8$.
- **Find** y **Erase** son $O(n)$ en las cuatro, como se esperaba: guardar **tail** o **prev** no ayuda a encontrar un valor arbitrario.

Efectos que no son de complejidad. Dos fenómenos aparecen en las gráficas y conviene explicarlos para no confundirlos con Big-O:

1. **Salto entre 10^6 y 10^7 .** Los métodos $O(n)$ se multiplican por entre $\times 15$ y $\times 27$ en esa década, y no por 10. Con 10^6 nodos (unos 24 MB) la lista todavía cabe en buena parte en la memoria caché del procesador; con 10^7 (unos 240 MB) ya no, y cada salto a **next** puede esperar a la memoria RAM, que es mucho más lenta. Entre 10^7 y 10^8 la lista ya no cabe en ningún caso

y el crecimiento vuelve a ser $\times 10$. Es un cambio en la *constante* (de 2–5 ns a 6–9 ns por nodo recorrido), no en la complejidad.

2. *Algunos métodos $O(1)$ suben hasta $\approx 3\mu s$ y luego se estabilizan.* Es el caso de **AddAfter/AddBefore** en las dobles y de **PushBack** en la simple con cola. Justo antes de medirlos, la preparación o el deshacer de la repetición anterior recorren la lista entera (el **find** para obtener el nodo, el **popBack** $O(n)$ que deshace el **pushBack**). Ese recorrido saca de la caché todo lo que la operación medida necesita, que entonces sufre unos pocos fallos de caché: un costo fijo de unos microsegundos. La prueba de que es constante es la última década: de 10^7 a 10^8 esos métodos cambian entre $\times 0,25$ y $\times 1,13$, mientras que un método $O(n)$ cambiaría $\times 10$. La doble con cola en **PushBack**, cuyo deshacer es $O(1)$ y no ensucia la caché, se mantiene plana en $0,046\mu s$.

Constantes entre variantes (H1). Entre implementaciones con la misma Big-O las diferencias son pequeñas. En los métodos $O(1)$ son de pocos nanosegundos y quedan por debajo de la resolución (figura 5, paneles PushFront, PopFront, TopFront). En los métodos $O(n)$, como **Find**, varían hasta un 30 % para $n = 10^8$ (0,72 s en la simple sin cola frente a 0,94 s en la doble con cola). Esa diferencia depende de cómo quedaron los nodos en memoria y no de la estructura del algoritmo: en la JVM usada (64 bits con referencias comprimidas) un nodo simple ocupa 20 bytes que se redondean a 24, y uno doble exactamente 24, así que ambos ocupan lo mismo. La H1 se confirma.

4.3 Parte 2: MyStack y MyQueue con arreglo circular

Cuadro 7: ArrayStack (implementa MyStack): tiempo promedio (μs).

n	Push	Pop	Peek	IsEmpty	Size	Delete
10^1	0,089	0,108	0,191	0,161	0,167	0,262
10^2	0,043	0,093	0,161	0,141	0,141	0,629
10^3	0,043	0,105	0,160	0,141	0,143	4,364
10^4	0,043	0,056	0,079	0,072	0,067	47,4
10^5	0,043	0,047	0,052	0,051	0,052	472,0
10^6	0,067	0,073	0,143	0,122	0,095	4 826
10^7	0,065	1,653	1,472	0,523	0,413	46 818
10^8	0,079	0,045	0,065	0,049	0,052	459 725
Teórico	$O(1)^*$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Razón $\times 10$	$\times 1,2$	$\times 0,0$	$\times 0,0$	$\times 0,1$	$\times 0,1$	$\times 9,8$
Empírico	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$

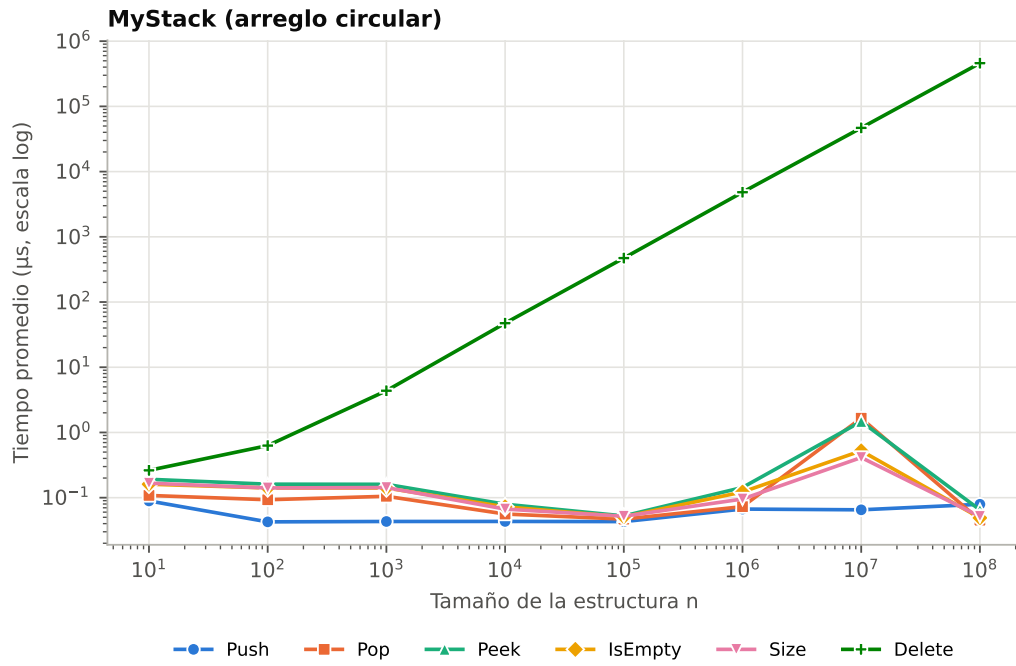


Figura 6: Métodos de la pila sobre arreglo circular dinámico.

Cuadro 8: ArrayQueue (implementa MyQueue): tiempo promedio (μ s).

n	Enqueue	Dequeue	Front	IsEmpty	Size	Delete
10^1	0,090	0,100	0,051	0,144	0,146	0,246
10^2	0,042	0,045	0,051	0,141	0,145	0,528
10^3	0,043	0,045	0,051	0,142	0,148	3,216
10^4	0,043	0,046	0,052	0,071	0,118	31,7
10^5	0,055	0,051	0,061	0,059	0,133	588,5
10^6	0,042	0,045	0,052	0,052	0,115	3 507
10^7	0,044	0,164	0,127	0,185	0,102	33 238
10^8	0,047	0,224	0,164	0,052	0,050	360 588
Teórico	$O(1)^*$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Razón $\times 10$	$\times 1,1$	$\times 1,4$	$\times 1,3$	$\times 0,3$	$\times 0,5$	$\times 10,8$
Empírico	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(n)$

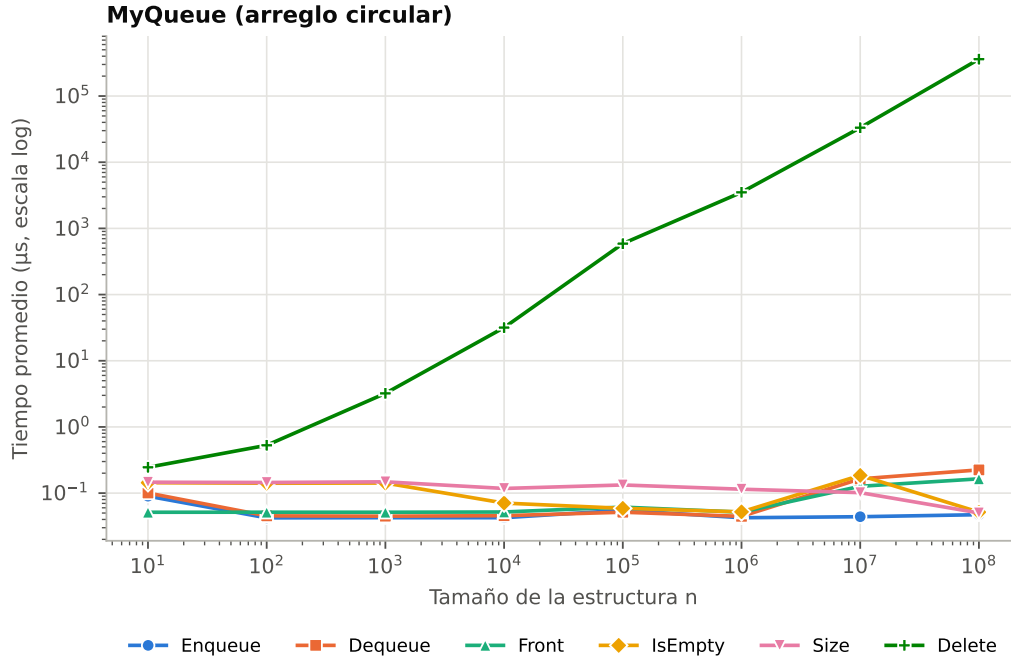


Figura 7: Métodos de la cola sobre arreglo circular dinámico.

Teórico vs. empírico. De nuevo coinciden. `push`, `pop`, `peek`, `enqueue`, `dequeue`, `front`, `isEmpty` y `size` se mantienen entre 0,04 y 0,22 μs de $n = 10$ a $n = 10^8$ (salvo un pico aislado en 10^7 , comentado abajo): $O(1)$. `delete` crece exactamente $\times 10$ por década ($\times 9,8$ en la pila, $\times 10,8$ en la cola): $O(n)$. En $n = 10^7$ varios métodos $O(1)$ de la pila muestran un pico aislado de 0,4–1,7 μs . No se repite en 10^8 , así que corresponde a interferencia puntual del sistema (con solo 10 repeticiones en ese tamaño, una pausa pesa en el promedio), no a crecimiento.

`delete` en la pila (0,46 s para 10^8) es un 27 % más lento que en la cola (0,36 s). Esto concuerda con los casos medidos: en la pila el valor está en la base, así que se recorren los n elementos y además se desplazan $n - 1$; en la cola el valor está al final y solo se recorre. En ambos casos el trabajo es proporcional a n .

4.3.1 Análisis del crecimiento del arreglo dinámico

Para analizar la estrategia de crecimiento se hicieron dos experimentos adicionales (tabla 9 y figura 8):

1. **Costo total de n inserciones** desde una pila vacía de capacidad 8, dividido entre n . Con la estrategia de **duplicar** el costo por inserción se mantiene entre 4,5 y 6 ns desde $n = 10^3$ hasta $n = 10^8$: es $O(1)$ amortizado, como predice el análisis de la sección 2.3. Este número es además la mejor estimación del costo real de un `push`, sin el piso de 40–70 ns que impone medir operaciones sueltas. Con **incremento fijo de +1000** el costo por inserción crece con n y el total se multiplica por $\times 98$ entre 10^5 y 10^6 (cuadrático). Para $n = 10^6$ tarda 424 ms, **95 veces más** que duplicando (4,5 ms). Para 10^7 tardaría varios minutos, así que no se midió. Se confirma la H3.
2. **Un solo push con el arreglo lleno.** Es la inserción que dispara la copia. Crece linealmente con n y llega a 163 ms para $n = 10^8$, frente a 0,08 μs de un `push` normal. Esta es la otra cara del costo amortizado: el promedio es excelente, pero una operación puntual puede ser millones de veces más lenta que las demás.

Cuadro 9: Inserción de n elementos desde vacío: tiempo total y costo por inserción según la estrategia de crecimiento.

n	x2: total	x2: ns/push	+1000: total	+1000: ns/push
10^1	0,132 μ s	13,2	0,760 μ s	76,0
10^2	0,616 μ s	6,2	1,061 μ s	10,6
10^3	4,677 μ s	4,7	3,445 μ s	3,4
10^4	50,8 μ s	5,1	69,0 μ s	6,9
10^5	450,4 μ s	4,5	4 303 μ s	43,0
10^6	4 459 μ s	4,5	423 563 μ s	423,6
10^7	52 721 μ s	5,3	—	—
10^8	587 541 μ s	5,9	—	—

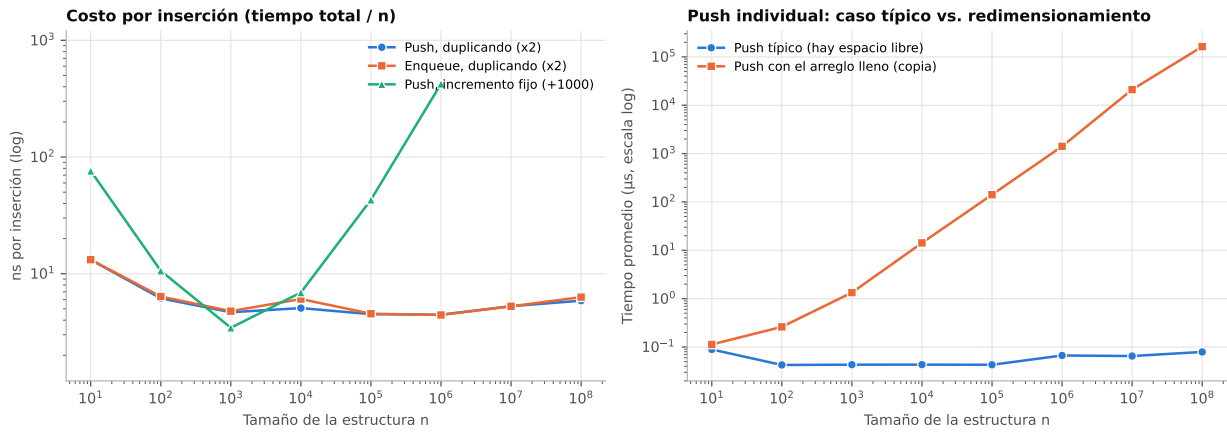


Figura 8: Izquierda: costo amortizado por inserción (duplicar vs. incremento fijo). Derecha: push típico vs. push con el arreglo lleno.

4.4 Parte 3: comparativa de métodos equivalentes List / (Stack, Queue)

Una pila se puede implementar con una lista usando solo el inicio: `push = pushFront`, `pop = popFront`, `peek = topFront`. Una cola, insertando al final y sacando del inicio: `enqueue = pushBack`, `dequeue = popFront`, `front = topFront`. Además, `delete` equivale a `erase`, e `isEmpty/size` a `Empty/Size`.

Criterio para elegir la implementación de List. Para cada método se eligió la variante más óptima en dos pasos: (1) la de **mejor complejidad teórica**; (2) entre las empatadas, la **más liviana**, es decir, la que mantiene menos referencias por operación (simple antes que doble, sin cola antes que con cola cuando la cola no se usa). El paso (2) no se basa en los tiempos medidos porque, como mostró la parte 1, las variantes empatadas difieren en pocos nanosegundos, por debajo de la resolución, y elegir por ese ruido no tendría sentido.

- **PushFront, PopFront, TopFront** (vs. `push`, `pop`, `peek`, `dequeue`, `front`): son $O(1)$ en las cuatro variantes. Se elige la **simple sin cola**, porque en cada operación solo actualiza `head`. Las otras tres, además, tendrían que mantener `tail` o `prev`, que una pila no necesita.
- **PushBack** (vs. `enqueue`): solo las variantes con cola son $O(1)$. Entre ellas se elige la **simple con cola**: es la mínima estructura que da una cola eficiente (`pushBack` $O(1)$ por `tail`, `popFront` $O(1)$ por `head`), y no necesita `prev` porque una cola nunca saca por el final.
- **Erase** (vs. `delete`): es $O(n)$ en las cuatro. Se elige la **simple sin cola**, la más sencilla.
- **Empty, Size**: $O(1)$ en todas gracias al contador; **simple sin cola**.

Cuadro 10: Métodos equivalentes: implementación de lista elegida y tiempo para $n = 10^8$.

Lista	Implementación elegida	Arreglo circular	Big-O (L / A)	Lista (μ s)	Arreglo (μ s)
PushFront	SLL sin cola	ArrayStack.Push	$O(1) / O(1)^*$	0,041	0,079
PopFront	SLL sin cola	ArrayStack.Pop	$O(1) / O(1)$	0,043	0,045
TopFront	SLL sin cola	ArrayStack.Peek	$O(1) / O(1)$	0,050	0,065
PushBack	SLL con cola	ArrayQueue.Enqueue	$O(1) / O(1)^*$	3,087	0,047
PopFront	SLL sin cola	ArrayQueue.Dequeue	$O(1) / O(1)$	0,043	0,224
TopFront	SLL sin cola	ArrayQueue.Front	$O(1) / O(1)$	0,050	0,164
Erase	SLL sin cola	ArrayStack.Delete	$O(n) / O(n)$	595 694	459 725
Erase	SLL sin cola	ArrayQueue.Delete	$O(n) / O(n)$	595 694	360 588
Empty	SLL sin cola	ArrayStack.IsEmpty	$O(1) / O(1)$	0,064	0,049
Size	SLL sin cola	ArrayStack.Size	$O(1) / O(1)$	0,068	0,052

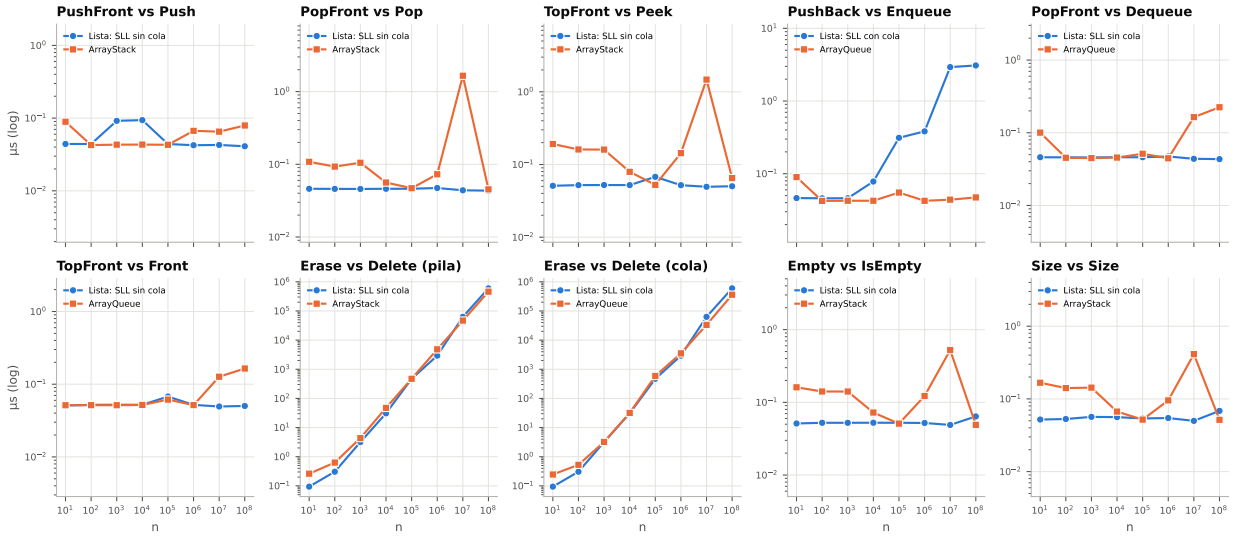


Figura 9: Lista (implementación elegida) vs. arreglo circular, método por método.

Resultados.

- **Operaciones $O(1)$ (push/pop/peek, dequeue/front, isEmpty/size):** empatan. Ambas estructuras quedan en 0,04–0,2 μ s en todo el rango, sin tendencia (salvo el pico aislado de la pila en 10^7). Con la resolución disponible no se puede declarar un ganador: las dos cumplen $O(1)$ y la diferencia real es de unos pocos nanosegundos.
- **PushBack vs. enqueue:** ambos son $O(1)$ y ambos quedan planos en la última década. La lista marca $\approx 3 \mu$ s para $n \geq 10^7$ por el efecto de caché explicado en la parte 1: su deshacer (**popBack** $O(n)$) ensucia la caché antes de cada medición. Aun así hay una diferencia estructural real. Cada **pushBack** crea un objeto nodo nuevo, que el recolector de basura tendrá que gestionar, mientras que **enqueue** escribe una referencia en una posición ya reservada del arreglo.
- **Erase vs. delete:** los tres son $O(n)$ y sus curvas son paralelas (figura 9). Para n pequeño la lista es algo más rápida, porque **delete** además desplaza elementos. Para n grande el arreglo gana a pesar de hacer más trabajo: para $n = 10^8$, 0,46 s en la pila (que recorre y desplaza) y 0,36 s en la cola, frente a 0,60 s en la lista. Por elemento, el arreglo procesa $\approx 3,6$ ns en la cola (solo búsqueda) y la lista ≈ 6 ns por nodo. El arreglo es memoria contigua que el procesador lee por bloques y anticipa; la lista obliga a seguir una referencia a una posición arbitraria por cada nodo. Se confirma la H2.

5 Conclusiones

5.1 Cuándo usar cada estructura

- **Lista simple sin cola:** cuando solo se opera por un extremo, es decir, como pila o como lista que se recorre de principio a fin. Es la más liviana en código y en referencias por nodo.
- **Lista simple con cola:** cuando se inserta por el final y se saca por el inicio (cola FIFO). Si también se necesita sacar por el final, no sirve: `popBack` sigue siendo $O(n)$.
- **Lista doble sin cola:** poco útil por sí sola. Paga el costo de mantener `prev` y solo gana `addBefore` $O(1)$, pero sigue siendo $O(n)$ en todo lo que ocurre al final.
- **Lista doble con cola:** la única con todas las operaciones de los extremos en $O(1)$ (sirve como *deque*) y con inserción y eliminación $O(1)$ en cualquier punto del que ya se tenga la referencia. Es la opción cuando se insertan o borran elementos en medio de una secuencia, por ejemplo en listas de reproducción, editores de texto o una caché LRU (lista doble más tabla hash).
- **Pila y cola con arreglo circular dinámico:** la mejor opción general para pilas y colas. Tienen la misma complejidad que la mejor lista, pero usan mucha menos memoria (4 a 8 bytes por elemento contra 24 de un nodo), no crean objetos por cada inserción y recorren más rápido por la localidad de memoria. Su debilidad es la latencia puntual al crecer (163 ms para 10^8). Si el tamaño máximo se conoce de antemano, se evita reservando la capacidad desde el constructor.
- **Nunca** conviene hacer crecer un arreglo dinámico en una cantidad fija: convierte n inserciones en $O(n^2)$.

5.2 Pilas y colas en aplicaciones reales

- **Pilas (LIFO):** la pila de llamadas de cualquier programa (cada llamada a un método apila su contexto y el `return` lo desapila; por eso la recursión infinita causa `StackOverflowError`); deshacer/rehacer en editores; el botón “atrás” del navegador; verificación de paréntesis balanceados y evaluación de expresiones en compiladores y calculadoras; recorrido en profundidad (DFS) y *backtracking*, como en laberintos o sudokus.
- **Colas (FIFO):** colas de impresión y de tareas en sistemas operativos; atención de solicitudes en servidores web y *pools* de hilos; colas de mensajes entre servicios; búferes de audio y video en *streaming*, donde el arreglo circular de tamaño fijo es precisamente la estructura estándar (*ring buffer*); recorrido en anchura (BFS), por ejemplo para calcular la ruta con menos transbordos en un sistema de transporte.

5.3 Arreglos dinámicos vs. listas enlazadas en Java

	Arreglo dinámico			Lista enlazada
Acceso por posición	$O(1)$			$O(n)$
Insertar/borrar en extremos	$O(1)$ amortizado (con arreglo circular)	(con arreglo		$O(1)$ (con cola y, para <code>popBack</code> , doble)
Insertar/borrar en medio	$O(n)$ (desplazar)			$O(1)$ si ya se tiene el nodo
Memoria por elemento	4-8 bytes (referencia más espacio libre)	referencia más espacio		24 bytes por nodo más la referencia
Recorrido	Rápido: memoria contigua, caché			Lento: un salto de memoria por nodo
Recolector de basura	Un solo objeto grande			Un objeto por elemento
Peor caso puntual	Copia $O(n)$ al crecer			Estable, sin picos por crecimiento

En la práctica, en Java el arreglo dinámico gana en la gran mayoría de los casos. Por la misma razón, la documentación de Java recomienda `ArrayDeque` (un arreglo circular dinámico, como el de este trabajo) antes que `LinkedList` para implementar pilas y colas. La lista enlazada se justifica cuando hay muchas inserciones o eliminaciones en medio de la secuencia a partir de referencias ya conocidas, o cuando no se pueden tolerar los picos de latencia de un redimensionamiento.

La lección más general del trabajo es que Big-O predijo correctamente cómo *escala* cada método, en los 60 casos medidos, pero no dice nada de las *constantes*. La memoria caché, el recolector de basura y la resolución del reloj cambiaron los tiempos por factores de 2 a 25 sin alterar la clase de complejidad. Para decidir entre dos estructuras con la misma Big-O hay que medir, y medir bien: una sola operación a la vez, en el peor caso, con calentamiento, repeticiones y sin mezclar la medición con otras tareas.